

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В.Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)

Институт информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) образовательной программы Разработка программно-информационных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

**Реализация алгоритма извлечения звука
музыкального инструмента из звукового потока
на основе методов искусственного интеллекта**

Студент (ка): Барышникова Варвара Дмитриевна

Зав. кафедрой: канд. техн. наук, доц. Поляков Владимир Михайлович

Руководитель: канд. физ.-мат. наук, доц. Осипов Олег Васильевич

К защите допустить

Зав. кафедрой _____ /Поляков В.М./

« _____ » _____ 2026 г.

Белгород 2026 г.

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В.Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)

Институт информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) образовательной программы Разработка программно-информационных систем

Утверждаю:

Зав. кафедрой _____

«_____» _____ 2026 г.

ЗАДАНИЕ

на выпускную квалификационную работу студента (ки)

Иванова Ивана Ивановича

1. Вид выпускной квалификационной работы (ВКР): *бакалаврская работа*.
2. Тема ВКР *Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения* утверждена приказом по университету от «28» января 2026 г. №2/124.
3. Срок сдачи студентом (кой) законченной ВКР: «12» июня 2026 г.
4. Исходные данные: ключевые слова, название алгоритмов и методов, технологии разработки и т.п. Пример: рекомендательные системы, подходы построения рекомендательных систем, методы матричной факторизации, оптимизационные методы обучения модели, аффинные преобразования, трёхмерная графика, машинное обучение, клиент-серверное приложение, генеративная нейронная сеть.
5. Содержание ВКР: Здесь должны быть перечислены названия разделов (глав) пояснительной записки. Пример:
 - Введение,
 - Описание предметной области прогнозирования погоды,
 - Разработка архитектуры программного обеспечения прогнозирования погоды,
 - Разработка алгоритмов и программного обеспечения прогнозирования погоды,
 - Тестирование программной системы прогнозирования погоды,
 - Руководство пользователя разработанной системы прогнозирования погоды,
 - Заключение,
 - Список литературы,
 - Приложение А,
 - Приложение Б.

6. Перечень графического материала: 1278 рисунков, 1785 таблиц. Презентация включает: постановку задачи, актуальность, технологии, численные эксперименты, внешний вид программы, заключение.

Консультанты по работе с указанием относящихся к ним разделов:

Раздел	Консультант	Задание выдал (подпись, дата)	Задание принял (подпись, дата)

Дата выдачи задания: «17» января 2026 г.

(подпись руководителя)

Задание принял (приняла) к исполнению

(подпись студента (ки))

КАЛЕНДАРНЫЙ ПЛАН

№ п/п	Наименование этапов работы	Срок выполнения этапов работы	Примечание
1	Анализ предметной области. Постановка задачи разработки системы прогнозирования погоды методами машинного обучения.	18.01.26 – 15.02.26	Выполнено
2	Разработка алгоритмов, структур нейросетей и архитектуры системы прогнозирования погоды.	15.02.26 – 20.03.26	Выполнено
3	Разработка программной части системы прогнозирования погоды.	20.03.26 – 29.03.26	Выполнено
4	Разработка клиентской части системы прогнозирования погоды.	29.03.26 – 29.04.26	Выполнено
5	Тестирование программного обеспечения для прогнозирования погоды.	29.04.26 – 17.05.26	Выполнено
6	Оформление пояснительной записки. Подготовка выступления.	17.05.26 – 10.06.26	Выполнено
5	Тестирование программного обеспечения для прогнозирования погоды.	29.04.26 – 17.05.26	Выполнено
6	Оформление пояснительной записки. Подготовка выступления.	17.05.26 – 10.06.26	Выполнено

Студент (ка) _____ *Иванов Иван Иванович*
(подпись) (Ф.И.О.)

Руководитель _____ *Осипов Олег Васильевич*
(подпись) (Ф.И.О.)

Результаты проверки ЭВ ВКР на заимствование

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Студент (ка) Иванов И.И. ПВ-212 16.06.2026
(Ф.И.О.) (подпись) (группа) (дата)

Тема ВКР: Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения.

ВКР прошла проверку на объём заимствований.

Итоговая оценка заимствований: **3,1415926535897932384626433832%.**

Работу проверил _____ 16.06.2026 Хлопов А.М.
(подпись) (дата) (Ф.И.О.)

Руководитель ВКР

канд. соц. наук, доцент	16.06.2026	Островский А.М.
(уч. степень, должность)	(подпись)	(Ф.И.О.)

ОПРЕДЕЛЕНИЯ, СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ

Стем (от англ. stem – «основа») – единица музыкального аудиопотока в задаче извлечения источников. Представляет собой запись отдельного типа музыкального инструмента или вокала в исходной многокомпонентной музыкальной аудиозаписи.

SiSec 2018 (Signal Separation Evaluation Campaign) – кампания по оценке разделения сигналов 2018 года.

SiSec 2018 MUS – задача, которая решалась во время кампании SiSec 2018. Задача состояла в оценке систем, которые извлекают музыкальные инструменты из популярных музыкальных аудиопотоков, миксов.

MUSDB18 – датасет музыкальных композиций с разделенными стемами, который был составлен в рамках задачи SiSec 2018 MUS.

MSS (music source separation) – задача выделения источников из музыкального потока.

SOTA (state of art) – ”состояние искусства стандарт, признанный в исследовательской среде в рамках какой-либо исследовательской задачи.

MSST-WebUI (Music-Source-Separation-Training WebUI) – веб-интерфейс, который объединяет множество современных моделей разделения источников.

ПО – программное обеспечение.

СУБД – система управления базами данных.

ООП – объектно-ориентированное программирование.

ЭВМ – электронно-вычислительная машина.

КОП – код операции.

АЛУ – арифметико-логическое устройство.

ПЛИС – программируемая логическая интегральная схема.

БПФ – быстрое преобразование Фурье.

СЛАУ – система линейных алгебраических уравнений.

ОС – операционная система.

API (Application Programming Interface) – программный интерфейс, обеспечивающий взаимодействие между компонентами программного обеспечения.

RGB (red, green, blue) – цветовой формат машинной графики, основанный на смешении компонентов красного, зелёного и синего цветов.

WWW (World Wide Web) – всемирная паутина, система взаимосвязанных гипертекстовых документов.

PDF (Portable Document Format) – переносимый формат электронных документов.

HTML (HyperText Markup Language) – язык гипертекстовой разметки, который используется для создания и структурирования веб-страниц.

SQL (Structured Query Language) – язык структурированных запросов, используемый для работы с базами данных.

VPN (Virtual Private Network) – виртуальная частная сеть, обеспечивающая защищённое соединение через интернет.

IoT (Internet of Things) – интернет вещей, концепция подключения различных устройств к интернету для обмена данными.

JSON (JavaScript Object Notation) – текстовый формат обмена данными, который используется для хранения и передачи структурированной информации между системами и приложениями.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

- Актуальность задачи извлечения инструментов
- Цель работы: адаптация модели Demucs для улучшения качества извлечения гитары
- Задачи исследования

Актуальность (музыкальное ADR, мастеринг, ремиксы, авторские права и т.п.).

Объект, предмет, цель, задачи, научная новизна.

Задача: Адаптация лёгких архитектур для извлечения гитары и пианино из полифонических композиций с учётом спектрального перекрытия (Частотно-временное перекрытие) и последующего восстановления тембрового качества.

1 АНАЛИЗ СОСТОЯНИЯ ИССЛЕДОВАНИЯ В ОБЛАСТИ ИЗВЛЕЧЕНИЯ ИСТОЧНИКОВ

Выделение аудио-источников (Audio Source Separation) — процесс разделения индивидуальных источников звука в смешанном аудиосигнале. Цель такой задачи — выделить конкретные источники, например вокал, инструменты или фоновый шум, из сложной аудиозаписи (Рисунок ??).

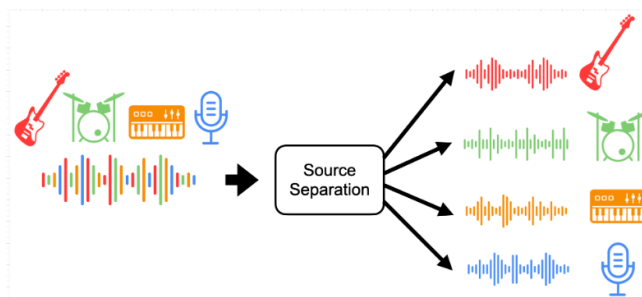


Рисунок 1.1 – Упрощенный процесс разделения источников из аудиозаписи.

Источник

Для разделения используются алгоритмы обработки сигналов, которые используют различные техники: спектральный анализ, анализ во временной и частотной областях, машинное обучение.

Итогом выполнения задачи декомпозиции смешанного аудиосигнала является входные данные в виде составляющих компонент, называемых стемами (дорожками) [2]. Стем (от англ. stem – «основа») является единицей музыкального аудиопотока. Когда происходит запись в студии звукозаписи, например, музыкального трека, то происходит запись отдельных музыкальных инструментов и вокала (стемов) и образуется финальная композиция, песня. И цель задачи извлечения сигналов, в том, чтобы воссоздать эти индивидуальные стеми из смешанного сигнала-музыкальной дорожки. [3]

Притом, такая задача подразумевает получение из одного аудиотрека (аудиопотока) несколько источников (то есть задача сложнее, чем из потока шума получить какой-то один источник – в статье [3] приводится пример появления задачи – потребность услышать и вычленив из общего шума коктейльной вечеринки голос и рассказ своего собеседника). Тогда можно считать, что исходный музыкальный трек состоит из стемов, и каждый из них можно извлечь из него.

Поэтому необходимо провести обзор методов разделения музыкальных источников, а также методы дообучения нейросетей.

1.1 Вклад кампании SiSEC MUS 2018 в задачу разделения источников

В статье о реализации нейронной сети Open-Unmix [2] говорится про кампанию по оценке разделения сигналов 2018 года (SiSEC 2018 – Signal Separation Evaluation Campaign) [4] [5]. В рамках этой кампании решалась задача SiSEC MUS (оценка систем, которые извлекают музыкальные инструменты из популярных музыкальных аудиопотоков, миксов (mix – смесь)) такие стемы были сгруппированы в общие категории :

- барабаны (drums),
- бас (bass),
- вокал (vocals),
- другие музыкальные инструменты и звуки (other).

В результате кампании SiSEC Mus 2018 был выпущен датасет MUSDB18 [6] [7], содержащий 150 музыкальных композиций с разделенными стемами в сумме составляющие 10-часовой плейлист. MUSDB18 и по сей день является самым большим бесплатным датасетом в открытом доступе и активно используется в задачах дообучения нейронных сетей, решающих задачу разделения источников [2] (при необходимости или скудности материалов этого датасета исследователи прибегают к самостоятельному сбору специфичных датасетов [8]).

1.2 Способы представления звука для задачи разделения источников

1.3 Обзор архитектур разделения источников

1.3.1 Классические методы source separation

ICA

NMF

спектральные методы

1.3.2 Современные нейронные архитектуры

Современные архитектуры MSS (music source separation) можно условно разделить на два основных класса в зависимости от того, в какой области они обрабатывают аудиосигнал: во временной области (Demucs [3]) или в частотной (Open-Unmix [2] и Spleeter). Далее рассмотрим эти основные виды архитектур и сравним их способы реализации и применение.

Архитектуры, которые представляют собой SOTA (state of art)////////////////////////////////////

U-Net на спектрограммах

Conv-TasNet-подобные

Open-Unmix Существуют модели, работающие в частотной области, такие как Open-Unmix [?] и Spleeter. Такие модели сначала преобразуют аудиосигнал в спектрограмму с помощью быстрого преобразования Фурье (БПФ), где каждый пиксель представляет собой магнитуду (амплитуду) определенной частоты в определенный момент времени. Обработка происходит над этой спектрограммой, после чего полученная маска применяется к исходной спектрограмме для извлечения целевого источника, а затем выполняется обратное преобразование Фурье для получения результирующей аудиодорожки.

Архитектура модели Open-Unmix описана в статье [?] (Рисунок ??).

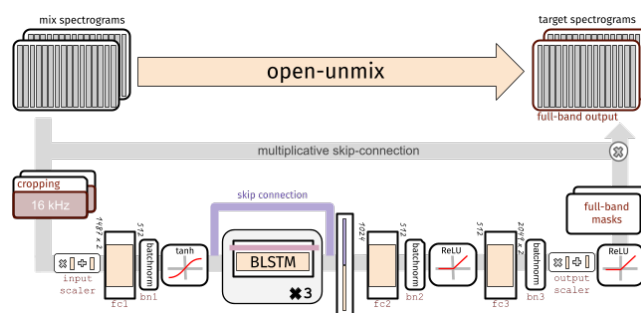


Рисунок 1.2 – Архитектура модели Open-Unmix [?]

Общая идея модели Open-Unmix заключается в преобразовании исходного файла смеси (микса) музыкальных инструментов в формате спектрограммы (mix spectrograms) в набор спектрограмм отделенных музыкальных инструментов из этого микса (target spectrograms).

Модель Open-Unmix состоит из следующих компонентов:

- 1) Подготовка входных данных ("Cropping 16 kHz" "Input Scaler") – на этом этапе происходит дробление исходного аудио (разделение по 16КГц) и нормализация входных спектрограмм для улучшения качества работы модели.
- 2) Энкодер (кодировщик) – это отдельная модель внутри Open-Unmix, которая занимается сжатием входного аудиопотока в компактный набор признаков. В свою очередь модель энкодера состоит из следующих блоков:
 - **fc1 (Fully Connected)** – слой нейросети, который начинает анализировать данные.
 - **bn1 (Batch Normalization)** – это слой нейронной сети, который реализует операцию (алгоритм) нормализации. bn1 обозначает первый слой пакетной нормализации (Batch Normalization layer) в архитектуре. Это обучаемый блок, который стабилизирует распределение активаций внутри мини-батча, ускоряет сходимость обучения и часто выступает как

мягкий регуляризатор. В схеме он расположен между полносвязным слоем $fc1$ и нелинейной функцией активации $\tanh$.

- **$\tanh$** функция гиперболического тангенса – нелинейная функция активации, которая решает, какие сигналы пропустить дальше, а какие остановить. Формула функции гиперболического тангенса имеет вид:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.1)$$

3) Центральный модуль модели – блок на основе двунаправленной долговременной краткосрочной памяти (Bidirectional Long Short-Term Memory, BLSTM), выделенный на схеме фиолетовой рамкой. Данный компонент отвечает за моделирование нестационарных временных зависимостей музыкального сигнала. Блок BLSTM реализован в виде каскада из трёх последовательно соединённых рекуррентных слоёв (обозначение $\times 3$ на схеме). Многослойная архитектура обеспечивает иерархическое извлечение признаков: нижние слои кодируют низкоуровневые временные паттерны (огибающие, транзиенты), тогда как верхние слои агрегируют их в высокоуровневые семантические представления музыкальной структуры (мотивы, аккордовые прогрессии, тембральные характеристики). Для стабилизации обучения глубокой рекуррентной сети и компенсации проблемы затухания градиентов применяется механизм остаточного соединения (Skip Connection). Фиолетовая стрелка на схеме обозначает суммирование входного тензора с выходом блока BLSTM:

$$y = x + \mathcal{F}_{\text{BLSTM}}(x), \quad (1.2)$$

где x — входное представление, $\mathcal{F}_{\text{BLSTM}}(\cdot)$ — преобразование, выполняемое стеком BLSTM-слоёв. Данный приём обеспечивает прямой градиентный поток на всех этапах обратного распространения, сохраняет низкоуровневую спектральную информацию и ускоряет сходимость оптимизатора без риска деградации представлений при увеличении глубины сети.

4) Декодер – модуль, отвечающий за восстановление представлений, сжатых в энкодере. После извлечения временных зависимостей блоком BLSTM следует модуль декодирования, целью которого является восстановление размерности и структуры тензора, необходимого для последующего формирования маски. В данной конфигурации используются следующие элементы:

- Слои $fc2$ и $fc3$ (Fully Connected): Выполняют трансформацию скрытого вектора в пространство признаков, совместимое с размерностью вход-

ной спектрограммы.

- Слои bn2 и bn3 (Batch Normalization): Обеспечивают нормализацию активаций на промежуточных этапах декодирования, предотвращая смещение распределения признаков (covariate shift) перед нелинейностями.

- Функция активации ReLU (Rectified Linear Unit): В отличие от сигмоидальных функций, используемых ранее, на этапе декодирования применяется функция:

5) Мультипликативное маскирование – механизм. Прямая генерация спектрограммы «с нуля» часто приводит к артефактам и фазовым искажениям. Вместо этого модель вычисляет массивную маску (full-band mask) $\hat{M}$, которая определяет коэффициент присутствия целевого источника для каждого бина спектрограммы. Процесс разделения описывается формулой поэлементного умножения (Hadamard product):

$$\hat{\mathbf{S}} = \mathbf{X} \odot \hat{\mathbf{M}}, \quad (1.3)$$

где $\mathbf{X} \in R^{T \times F}$ — исходная входная спектрограмма смеси (суммарное представление всех инструментов), $\hat{\mathbf{M}} \in R^{T \times F}$ — предсказанная нейросетью маска (выход модуля декодирования после функции активации ReLU, обеспечивающей неотрицательность значений), $\hat{\mathbf{S}}$ — результирующая спектрограмма целевого источника (например, вокала).

Demucs

В статье «Music Source Separation in the Waveform Domain» разработчики соц сети Facebook описывают новый подход к методу source separation. Demucs (Hybrid Transformer Demucs) на данный момент является одним из стандартов индустрии (SOTA) задачи извлечения источников.

Demucs [3] – это нейронная сеть глубокого обучения, которая была разработана для решения задачи выделения аудио-источников. В основе нейросети Demucs лежит свёрточная нейронная сеть UNet [13], которая используется для семантической сегментации изображений. То есть Demucs использует подход сети Unet обработки изображений и применяет для обработки волн аудиосигнала по спектрограммам [3].

В статье музыкальная композиция сравнивается с коктейльной вечеринкой, где каждый голос, фоновая мелодия и шум – каждый этот элемент является стемом, и тогда задача выделения голоса собеседника, с которым ведется диалог на вечеринке, является как раз задачей разделения источников. Но задача выделения источников из музыкальной композиции отличается от задачи вы-

деления голоса собеседника на коктейльной вечеринке в первую очередь тем, что в ней интересует в этой задаче выделение не какого-то одного приоритетного источника звука из несвязанного фонового шума, а целый набор тонов и тембров в согласованном потоке.

Авторы утверждают, что на момент постановки проблемы этой статьи основные современные методы извлечения источника оперируют анализом спектрограмм, которые генерируются на основе кратковременного преобразования Фурье (short-time Fourier transform, STFT) [14] [15].

Метод основан на создании маски для magnitude spectrums for each frame and each source, and the output audio is generated by running an inverse STFT on the masked spectrograms reusing the input mixture phase.

Разработчики Demucs основываются на архитектуре Conv-Tasnet и адаптируют ее под извлечение источников (source separation).

Модели, работающие во временной области, такие как Demucs (сводная модель демиксера), обрабатывают аудиосигнал непосредственно в виде последовательности амплитудных значений (волн). Их главное преимущество заключается в сохранении полной информации о сигнале, включая фазовые характеристики, что критически важно для точного воспроизведения резких атак нот, характерных для многих инструментов, включая гитару и ударные .

В дальнейшем на основе базовой модели DEMUCS были разработаны Hybrid DEMUCS, Hybrid Transform DEMUCS и другие. Источники нужны

Spleeter

1.3.3 Ограничения существующих решений для извлечения конкретных инструментов

1.4 Методы дообучения нейронных сетей извлечения источников

1.4.1 Полное дообучение

Полное дообучение подразумевает обновление всех (или почти всех) весов предобученной модели на новом, целевом наборе данных.

В контексте моделей разделения источников звука (например, Open-Unmix) это означает повторное обучение всей сети на собственном датасете с дорожками гитары и пианино, с возможной корректировкой функции потерь и архитектурных деталей. Основное преимущество данного подхода — максимальная гибкость адаптации модели к специфике задачи, в том числе к особенностям частотного диапазона и перекрытию гармоник между инструментами. В то же время, полное дообучение требует значительных вычислительных ресурсов, высокого объема целевого датасета и аккуратного подбора гиперпарамет-

ров, поскольку существует риск переобучения и потери уже выученных обобщённых признаков.

Преимущества:

- 1) Максимальная гибкость: модель может глубоко адаптироваться к специфике частотных характеристик гитары и пианино, а также к стилям музыки, представленным в датасете.
- 2) На практике в ряде задач достигается наилучшее качество, так как все параметры оптимизируются под конкретную настройку.

Недостатки:

- 1) Высокие требования к вычислительным ресурсам: требуется много GPU-памяти и длительное время обучения, особенно для моделей с большим числом параметров.
- 2) Высокий риск переобучения при сравнительно малом объёме целевого датасета, а также риск разрушения уже выученных общих акустических признаков.

1.4.2 Методы «параметрического» дообучения (адаптация части параметров)

1. Adapters Вставляются маленькие модули-«адаптеры» между слоями сети; обучаются только они, а основные веса заморожены.

К параметрическим методам относятся подходы, при которых в структуру предобученной модели добавляются небольшие обучаемые модули, а основная часть весов остаётся замороженной. Например, в методе Adapters между слоями сети вставляются компактные преобразователи с низкой размерностью, которые и обучаются на целевой задаче. В аудио-задачах это позволяет сохранить уже выученные акустические признаки и при этом эффективно адаптировать модель к конкретным инструментам или доменам. Метод Prefix-tuning предполагает добавление обучаемого векторного префикса к входным представлениям, что также позволяет «настроить» поведение модели без изменения основных весов. Более радикальный подход — BitFit, при котором в процессе дообучения обновляются только биас-векторы (поправки смещения) слоёв, а все остальные параметры фиксируются; это приводит к крайне малому числу обучаемых параметров, но одновременно ограничивает способность модели сильно изменять своё поведение.

Плюс: экономия памяти, модульность (можно хранить несколько адаптеров под разные задачи).

2. Prefix-tuning / Soft prompts К скрытому представлению добавляется обучаемый префикс; встраивается в модель, не меняя её архитектуру.

Используется чаще в LLM, но идея «адаптации через добавленные параметры» переносима и в другие архитектуры.

3. BitFit Обучаются только биас-векторы (поправки смещения), а остальные веса заморожены.

Очень мало параметров, но адаптация ограничена.

1.4.3 Низкоранговые методы (LoRA и аналоги)

Низкоранговые методы дообучения основаны на идее замены части весов матриц на низкоранговые приращения, которые обучаемы, а исходные веса при этом остаются почти неизменными. В методе LoRA полная весовая матрица W аппроксимируется выражением $W + B A$ $W+BA$, где B B и A A — две низкоранговые матрицы (чаще всего гораздо меньшей размерности). В процессе дообучения корректируются только параметры B B и A A , что позволяет существенно снизить объём обучаемых параметров и требования к памяти GPU. В аудио-задачах, включая разделение звуковых дорожек, LoRA может применяться к отдельным слоям или компонентам модели (например, слоям свёртки или частотно-временным модулям), что обеспечивает эффективную адаптацию без полной пересборки сети. Разновидность LoRA — QLoRA — дополнительно использует квантование весов (например, до 4-bit представления), что позволяет дообучать даже более крупные модели на одной видеокарте и может быть полезной, если ресурсы в твоём эксперименте ограничены.

1. LoRA Замена части весов на сумму $W + B A$ $W+BA$, где B , A B,A — низкоранговая матрица; обучается только B , A B,A .

Эффективно для аудио- и языковых моделей, экономит память и время.

2. QLoRA LoRA + квантование весов (4-bit и т.п.), что позволяет дообучать даже крупные модели на одной видеокарте.

Хорошо подходит, если у тебя нет большого GPU-кластера.

1.5 Метрики оценки качества (SDR, SIR, SAR)

2 Разработка архитектуры программного обеспечения

- Архитектура Hybrid Transformer Demucs - Анализ ошибок базовой модели на тестовом наборе - Предложенные модификации: * Гитаро-специфичные аугментации * Дообучение на проблемных примерах * Фокусировка на критичных частотных диапазонах - Методика оценки результатов - спецификация скриптов?

архитектура программного решения, алгоритм выбора модели и результатах экспериментов

3 Разработка алгоритмов и программного обеспечения

- скрипты - Описание тестового набора данных - Результаты базовой модели Demucs - Результаты адаптированной модели - Сравнительный анализ метрик - Субъективная оценка качества

4 ЗАКЛЮЧЕНИЕ

- Основные результаты работы - Практическая значимость - Направления будущих исследований

5 ТРЕБОВАНИЯ К СОДЕРЖАНИЮ ПОЯСНИТЕЛЬНОЙ ЗАПИСКИ

Выпускная квалификационная работа (ВКР) программиста должна иметь прикладной и/или научно-исследовательский характер. Если работа имеет прикладной характер, в ней должны быть:

представлена постановка задачи и техническое задание на разработку программного обеспечения (ПО);

обоснована актуальность работы, выполнен обзор аналогов разрабатываемого ПО;

разработаны архитектура и программное обеспечение для решения поставленной задачи;

обеспечена информационная безопасность разработанного ПО;

проведено тестирование разработанного ПО и оценка его метрологических характеристик;

приведено руководство пользователя.

Если работа имеет научно-исследовательский характер, в ней должны содержаться:

научная новизна предложенных методов и алгоритмов;

актуальность работы;

постановка задачи в словесно-формульном виде;

математическая модель исследуемой предметной области;

описание предложенных математических методов и алгоритмов решения поставленной задачи (в виде формул, блок-схем);

результаты разработки программного обеспечения для решения поставленной задачи;

результаты проведения вычислительных экспериментов на реальных и/или модельных данных.

5.1 Тематики выпускных квалификационных работ

В ВКР обязательно должна присутствовать разработка программного обеспечения на каком-либо языке программирования. Обязательным результатом выполнения ВКР является готовое программное обеспечение. ВКР должна соответствовать одной из следующих тематик:

- 1) Системное программное обеспечение (драйверы, утилиты);
- 2) Прикладное программное обеспечение (редакторы, мультимедийные приложения);
- 3) Сетевое программное обеспечение;

- 4) Сервис-ориентированное программное обеспечение;
- 5) Разработка веб-приложений;
- 6) Разработка мобильных приложений;
- 7) Разработка нейросетевых приложений, в том числе с использованием глубокого обучения;
- 8) Разработка приложений искусственного интеллекта: компьютерное зрение, обработка естественного языка, обработка звука, генетические алгоритмы и др.;
- 9) Разработка приложений для решения задач математической логики и автоматизированного рассуждения;
- 10) Математическое программное обеспечение для решения задач моделирования различных процессов;
- 11) Разработка методов и алгоритмов вычислительной математики;
- 12) Разработка систем поддержки и принятия решений, а также нечёткой логики;
- 13) Разработка лексических и синтаксических анализаторов, компиляторов, интерпретаторов;
- 14) Разработка графических приложений, алгоритмов проективной геометрии;
- 15) Разработка игровых приложений;
- 16) Разработка программного обеспечения для микроконтроллеров, ПЛИС, встраиваемых систем;
- 17) Разработка программного обеспечения для анализа данных;
- 18) Разработка баз знаний, советающих и рекомендательных систем;
- 19) Разработка программного обеспечения для решения задач оптимизации, планирования, распределения, прогнозирования;
- 20) Разработка программного обеспечения с использованием параллельных вычислений;
- 21) Разработка программного обеспечения для автоматизации технологических процессов;
- 22) Разработка программного обеспечения для решения задач информационной безопасности.

5.2 Структура и содержание пояснительной записки

В работе не допускается наличие общеизвестного теоретического материала, доступного в литературе или в Интернете. Это снижает процент оригинальности работы, замедляет процедуры нормоконтроля и проверки заимство-

ваний. Достаточно вставить в текст работы ссылку на необходимый литературный источник. Допускается добавлять в работу теоретические сведения только в том случае, если последующие изложение без них невозможно. Не допускается включение теоретических сведений по основам программирования (любой парадигмы), описанию общеизвестных технологий и языков программирования, а также других базовых знаний, полученных в процессе обучения.

Во **введении** должно быть написано о том, какое место занимает именно заявленная проблема (задача) в области информационных технологий. Например, поставлена задача разработать алгоритм визуализации фигур вращения. Значит, нужно собрать из открытых источников информацию о том, какие успехи уже достигнуты другими инженерами при решении именно задач визуализации фигур вращения. Писать о том, что такое компьютерная графика, не нужно. Это касается не только введения – в пояснительной записке нужно писать **именно о своей задаче**. Не нужно «лить воду» и писать общие слова о том, как стремительно развивается область ИТ, до чего «довёл прогресс», что «современное человечество не может жить без компьютеров» и т.д. Таким образом необходимо обязательно отразить **актуальность решаемой задачи** и обосновать, почему потребовалось разрабатывать собственное ПО, а не использовать доступное. Далее приводятся цель и перечисляются задачи работы.

Цель работы – это то, ради чего разрабатывается программный продукт; ради чего проводится исследование. Это тот положительный эффект для конечного пользователя, который будет достигнут при использовании им нашей разработки. Например, мы разрабатываем клиент-серверное приложение для взаимодействия учеников и репетиторов. Цель такой работы – не разработка ПО, а ускорение (упрощение) взаимодействия ученика и репетитора.

Пример неправильно сформулированной цели

Разработка клиент-серверного приложения для взаимодействия ученика и репетитора.

Пример правильно сформулированной цели

Экономия времени и ускорение взаимодействия ученика и репетитора при проверке заданий и объяснении учебного материала с помощью клиент-серверного приложения.

Рассмотрим предыдущий пример. Предположим, мы разрабатываем ПО для визуализации тел вращения, которое должно работать на платформе Arduino.

Пример неправильно сформулированной цели

Разработка программного обеспечения для визуализации тел вращения на платформе Arduino.

Пример правильно сформулированной цели

Визуализация тел вращения в условиях ограниченных вычислительных ресурсов на платформе Arduino.

Задачи – последовательность шагов, которые нужно выполнить для достижения цели. К примеру, для задачи визуализации тел вращения можно выделить следующие задачи:

- 1) Поиск и сравнение существующих методов визуализации тел вращения неполигональными способами;
- 2) Разработка алгоритма визуализации тел вращения без использования полигонов с учётом освещения и затенения в условиях ограниченных вычислительных ресурсов;
- 3) Разработка архитектуры и программного обеспечения для визуализации тел вращения;
- 4) Тестирование разработанного программного обеспечения для визуализации тел вращения;
- 5) Создание руководства пользователя разработанного программного обеспечения для визуализации тел вращения.

Завершается введение кратким описанием структуры ВКР по главам. Объём введения – 3-4 стр.

В первой главе **«Анализ состояния исследований и разработок в предметной области...»** должны быть представлены: постановка задачи, актуальность работы, обоснование используемых технологий (стека технологий), описание (сравнение и обоснование) используемых методов, обзор существующих решений (аналогов) для решения поставленной задачи (подробный), краткий обзор предметной области.

Рекомендуется начинать первую главу с литературного обзора по заявленной теме: степень её разработанности, какие методы и алгоритмы уже предложены для решения поставленной задачи, какие у них есть достоинства и недостатки, со ссылками на литературные источники или материалы в Интернете. Если задача является новой, т.е. ранее не решалась, то можно писать о смежных задачах.

Если работа имеет ярко выраженный математический характер, то поста-

новка задачи должна быть выполнена в математической форме. В этом случае нужно написать в словесно-формульном виде, какие системы уравнений решаются с приведением всех переменных (с единицами измерения). Постановка прикладных задач может быть выполнена в словесной форме в виде перечисления методов, алгоритмов, архитектур, программ, интерфейсов и остальных компонентов, подлежащих разработке.

Теоретические сведения допускаются только в первой главе. Все последующие главы должны содержать только оригинальный материал и собственные разработки.

Рекомендуемая структура первой главы следующая:

1. Литературный обзор по разрабатываемой тематике,
2. Обзор существующих аналогов, методов и технологий разработки аналогичных программ,
3. Объект и предмет исследования,
4. Постановка задачи,
5. Практическая и теоретическая значимость,
6. Используемые методы и технологии разработки ПО,
7. Краткие теоретические сведения.

Рекомендуемый объём первой главы – 25-30 страниц.

Вторую главу **«Разработка архитектуры программного обеспечения»** рекомендуется начинать с написания технического задания (ТЗ) на разработку программного обеспечения. Само ТЗ при этом выносится в приложение А, а во второй главе указываются ссылки на него. В ТЗ должны быть изложены **все**

- функциональные требования к программному обеспечению,
- требования к интерфейсу,
- требования к надёжности,
- требования к используемому стеку технологий,
- требования к аппаратному обеспечению (железо),
- требования информационной безопасности.

ТЗ оформляется в соответствии с ГОСТ 19.201-78 [?].

Предполагается, что вторая глава пишется до непосредственной разработки программного обеспечения. Содержание этой главы не должно быть строго привязано к какой-либо технологии или языку программирования. В ней должна быть описана работа, которую обычно выполняет аналитик перед выдачей задачи программистам. Аналитик описывает взаимодействие модулей программного обеспечения на абстрактном уровне, не вникая глубоко в детали реализации на конкретных языках программирования. При описании архитектуры

требуется использовать стандартизированные методики формализации предметной области (нотации UML, IDEF0, DFD, ER-диаграммы, блок-схемы, диаграммы сценариев и другие).

Любое программное обеспечение должно быть декомпозировано, т.е. состоять из каких-то независимых друг от друга логических частей. Трудно представить себе сложную программу, которая состоит только из одной функции `main` внутри файла `main.cpp`, в которой реализован весь предъявляемый к ней функционал. Любая парадигма программирования предусматривает свой способ декомпозиции. В объектно-ориентированных языках (C++, Python, Java) ПО состоит из классов и методов, в процедурных (Си, Паскаль) – из подпрограмм, в функциональных (Haskell) – из чистых функций, в реляционных (SQL) – из таблиц, в языках описания электронных схем (Verilog) – из модулей. Даже в скриптовых языках текст делится на блоки. Во второй главе обязательно должна быть выполнена декомпозиция предметной области. Предположим, что в работе используется объектно-ориентированная парадигма программирования. В этом случае необходимо ввести подраздел **«Разработка структуры классов программного обеспечения»** и сначала описать в нём взаимодействие классов в виде диаграммы классов. Далее для каждого класса нужно написать вложенный подраздел, в котором перечислить, из каких подпрограмм он состоит и что они выполняют. Похожим образом выполняется декомпозиция и в других стилях программирования.

Во второй главе также могут быть представлены:

- разработка математических методов и алгоритмов (с использованием формул и блок-схем);
- оценка вычислительной сложности разработанных алгоритмов, возможные способы их распараллеливания и оптимизации;
- разработка структур данных (списков, матриц, графов и т.д.),
- разработка структур баз данных (в виде диаграмм типа «сущность-связь», ER-диаграмм),
- разработка структур нейронных сетей (в виде графов или послойно),
- разработка пользовательского интерфейса.

Рекомендуемый объём второй главы – 25-30 страниц.

Третья глава **«Разработка алгоритмов и программного обеспечения»** должна содержать подробное описание реализации программного обеспечения применительно к выбранным языкам и технологиям программирования. В этой главе должны быть приведены спецификации подпрограмм для каждого класса (модуля).

Пример спецификации подпрограммы

Спецификация функции `get_sum_and_max`.

1. Заголовок: `int get_sum_and_max(const int* a, int n, int* max)`.
2. Назначение: возвращает сумму элементов массива `a` длиной `n` и записывает по адресу `max` максимальный элемент массива `a`.
3. Входные параметры: `a` – указатель на массив, `n` – количество элементов массива.
4. Выходные параметры: `max` – указатель на максимальный элемент.

Если классов и подпрограмм много, то для специфицирования нужно выбрать основные. В третьей главе могут быть также представлены:

- описание структур данных на выбранном языке программирования;
- наиболее важные фрагменты программ с комментариями и пояснениями;
- описание предложенных оригинальных программных решений;
- программные решения для обеспечения информационной безопасности (хранение персональных данных и паролей, конфиденциальность и т.д.);
- краткое описание использованных сторонних библиотек;
- программные решения для оптимизации и распараллеливания разработанных алгоритмов;
- оценка производительности, отказоустойчивости, метрологических и эргономических характеристик;
- результаты тестирования;
- руководство пользователя;
- результаты внедрения;
- оценка качества разработанного ПО.

Приложения, осуществляющие хранение и обработку критически важной информации (включая аутентификационные данные и персональную информацию), должны разрабатываться в соответствии с требованиями безопасной разработки ПО и обеспечивать защиту информации от несанкционированного доступа и утечек.

Если руководство пользователя объёмное, то его рекомендуется вынести в отдельную главу.

Для научно-исследовательских задач должны быть приведены результаты вычислительных экспериментов на реальных и/или модельных данных.

Рекомендуемый объём третьей главы – 30-40 страниц.

По согласованию с дипломным руководителем, в ВКР могут быть добавлены дополнительные главы. После основных глав следует заключение и список использованной литературы. В заключении перечисляются прямые результаты разработки – преимущества разработанных алгоритмов и ПО над аналогами в плане скорости, функциональности, удобства и прочих характеристик.

5.3 Иллюстративный материал

К защите ВКР должен быть подготовлен иллюстративный материал в виде презентации, которая демонстрируется с помощью проектора. Презентация должна содержать:

1. Титульный лист (на нём должны быть указаны без сокращения ФИО студента и руководителя, должность и учёная степень руководителя);
2. Цель и задачи ВКР;
3. Актуальность работы и обоснование разработки;
4. Сравнительный анализ существующих аналогов;
5. Используемые технологии разработки, инструментальные, программные средства и библиотеки;
6. Постановка задачи на разработку ПО (для научно-исследовательских задач должна быть в словесно-формульном виде);
7. Описание обрабатываемых данных, их формат и структура;
8. Описание входных и выходных параметров программной системы;
9. Описание математических моделей, методов и алгоритмов;
10. Описание декомпозиции и архитектуры программного обеспечения (диаграммы классов, ER-диаграммы, диаграммы потоков данных, диаграммы пользовательских сценариев, графы и послойные схемы нейронных сетей, блок-схемы и прочее);
11. Описание системных требований (ОС, ПО, аппаратные требования);
12. Методы обеспечения информационной безопасности разработанного ПО (конфиденциальность, денежные операции, хранение персональных данных, хэширование паролей и т.д.);
13. Внешний вид программ;
14. Результаты моделирования, проведения вычислительных экспериментов;
15. Результаты тестирования, оценки производительности;
16. Заключение: основные выводы, достигнутые преимущества разработанного ПО.
17. Перечень публикаций по результатам выполнения ВКР.

По согласованию с дипломным руководителем в презентацию можно вклю-

чать и другие материалы ВКР. Не рекомендуется включать в презентацию фрагменты исходного текста программ, не имеющие большой смысловой нагрузки. Текст на слайдах должен быть написан чёрным цветом на светлом фоне, занимать всю доступную площадь и быть хорошо разборчивым, в том числе на картинках.

6 ТРЕБОВАНИЯ К ОФОРМЛЕНИЮ ПОЯСНИТЕЛЬНОЙ ЗАПИСКИ

Выпускная квалификационная работа (ВКР) может быть оформлена в текстовом редакторе или в издательской системе $\text{\LaTeX}$. Для автоматического форматирования ВКР под единые требования на кафедре ПОВТАС разработан стилевой файл `bstunote.sty` для текстового процессора $\text{\LaTeX}$. Данный файл совместим как с компилятором PDFLaTeX , так и XeLaTeX . Ниже представлены требования к оформлению ВКР и руководство по использованию пакета `bstunote`.

6.1 Параметры страницы

Выпускная квалификационная работа должна быть выполнена на листах формата А4. Для ВКР специалиста и бакалавра размеры полей должны быть заданы следующими:

- левое – не менее 27 мм,
- правое – около 10 мм,
- верхнее – около 10 мм,
- нижнее – не менее 21 мм.

Перечисленные параметры следует задавать стандартной командой из пакета `geometry`:

```
\usepackage[a4paper, left=30mm, right=9mm, top=9mm, bottom=24mm]{geometry}
```

Левое поле должно быть не менее 3 см, чтобы при прошивке обложка диплома не скрывала текст, как показано на рисунке ???. Если предполагается качественная прошивка, то левое поле можно уменьшить до 23 мм. Остальные поля нужно задать так, чтобы текст гармонично вписывался внутрь рамки. К примеру, отступить от рамки со всех сторон 3 мм. Чтобы посмотреть границы текста, нужно при подключении пакета `geometry` добавить опцию `showframe`.

Пакет `bstunote` автоматически подставляет «большие» рамки (по форме 2 из ГОСТ 2.104–2023 [?]) на первой странице каждого раздела и «маленькие» (по форме 2а) на следующих страницах. Здесь нижняя граница (`bottom`) увеличена до 24 мм, исходя из параметров чертёжного штампа по форме 2а, высота которого равна 15 мм.

Нужно учитывать, что положение рамки на бумаге после печати может отличаться от того, мы видим на экране. Чтобы оно совпадало, нужно обязательно задать в настройках масштаб печати, равный 100%. В некоторых pdf-просмотрщиках этот масштаб может называться «Реальный размер», «По размеру бумаги», «Фактический размер».

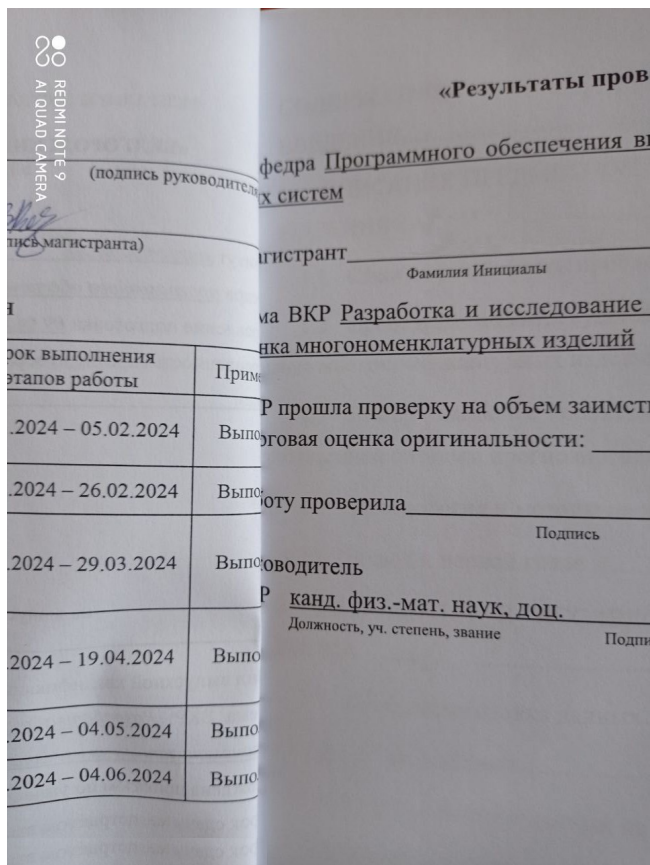


Рисунок 6.1 – Образец неправильно прошитого диплома. Авторский материал

Чтобы отключить рамки, нужно при подключении пакета `bstunote` сбросить флаг `drawstamps`:

```
\usepackage[drawstamps=false]{bstunote}
```

В магистерской диссертации рамки отсутствуют, а параметры полей рекомендуется задавать следующей командой:

```
\usepackage[a4paper, left=30mm, right=10mm, top=10mm,
bottom=10mm]{geometry}
```

6.2 Титульный лист, задание на ВКР, лист проверки заимствований

Начало пояснительной записки включает следующие листы:

- 1 лист – титульный лист, не нумеруется.
 - 2 лист – задание на ВКР, не нумеруется. Задание на ВКР печатается с двух сторон. То есть календарный план должен быть напечатан с обратной стороны.
 - 3 лист – результаты проверки электронной версии ВКР на заимствования, не нумеруется. Этот лист тоже обязательно вшивается в диплом.
- Далее следует сквозная нумерация, начиная с номера 4.
- 4 лист – лист определений, сокращений, обозначений. Этот лист необязательный. Он нумеруется цифрой «4».

Отзыв руководителя и справка о результатах проверки текста ВКР на наличие заимствований в системе АНТИПЛАГИАТ.ВУЗ вкладываются в уголок

на корочке диплома. В уголок магистерской диссертации также вкладывается рецензия от специалиста. Рецензия должна быть подписана и заверена печатью организации, которая её предоставила. Если уголок на корочке отсутствует, то его необходимо самостоятельно приклеить.

Нумерация страниц основного текста и приложений – сквозная (нумеруются все страницы, до последней страницы приложения), номер страницы указывается в штампе в графе «Лист».

6.3 Параметры основного текста

При использовании компилятора XeLaTeX разрешается использовать отечественные шрифты семейства PT Astra с засечками. Если используется компилятор PDFLaTeX, то можно использовать для вёрстки стандартные шрифты с засечками семейств Computer Modern или ParaType. Размер шрифта должен составлять 14 пунктов, но допускается его уменьшение до 13 или 12 пунктов. Размер шрифта задаётся в преамбуле при подключении базового класса `extarticle` следующей командой:

```
\documentclass[14pt]{extarticle}
```

Основной текст везде выравнивается по ширине. Красная строка равна 1,25 см, междустрочный интервал – полуторный. Эти параметры задаются автоматически. Абзацный отступ, шрифт и междустрочный интервал должны быть везде одинаковыми.

6.4 Параметры рубрикации

Заголовки структурных элементов (содержание, введение, основные главы, заключение, список литературы) выравниваются по левому краю с красной строки, печатаются прописными буквами и полужирным шрифтом. Данные структурные элементы начинаются с нового листа.

Нумеруются только основные главы. Точка после номера заголовка раздела (главы), подраздела второго и третьего уровней не ставится [?, раздел 6.5]. Если раздел или подраздел состоит из одного пункта, он также нумеруется. В конце заголовка точки не должно быть. Междустрочный интервал перед заголовком должен быть двойной, после заголовка – полуторный. Переносить слова по слогам в заголовках нельзя. Заголовки подразделов следует начинать с прописной буквы. Заголовки подразделов четвёртого уровня изначально не предусмотрены. При необходимости, следует писать их просто без номера полужирным шрифтом, не вынося в содержание. Подраздел не должен занимать меньше половины страницы. Если подраздел маленький, то рекомендуется объединить

его с разделом верхнего уровня. В конце каждой главы нужно написать подраздел «Вывод по n -ой (первой, второй, третьей и т.д.) главе» без нумерации и не вынося его в содержание.

Для создания заголовков нужно использовать стандартные команды:

```
\section*[ВВЕДЕНИЕ]{ВВЕДЕНИЕ} % Заголовок I уровня (глава) без номера
\section{ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ} % Первый уровень
\subsection{Методы решения нелинейных уравнений} % Второй уровень
\subsubsection{Метод Ньютона} % Третий уровень
\par\textbf{Математический вывод метода Ньютона} % Четвёртый уровень
```

Представленные команды секционирования уже настроены. Команда `\section` определена таким образом, что при её обнаружении во входящем потоке инициализируется процесс рисования рамок со штампами на всех страницах. Название текущего раздела при этом сохраняется и становится доступным через команду `\theStampSection`. Данная команда затем используется для автоматической подстановки в штампы названия текущего раздела (главы). В особых случаях, когда название главы не вмещается в графу штампа, можно вручную задать требуемый текст командой `\StampSection` после команды `\section`, указав в ней размер шрифта или добавив перенос по слогам в названии заголовка:

```
\StampSection{\fontsize{12.5pt}{10pt}\selectfont
ДЛИННЫЙ ЗАГОЛОВОК, КОТОРЫЙ НЕ ПОМЕЩАЕТСЯ В ГРАФУ, С УМЕНЬШЕННЫМ РАЗМЕРОМ
ШРИФТА ИЛИ ПЕ\ -РЕ\ -НО\ -СОМ ПО СЛО\ -ГАМ В НУЖНЫХ МЕСТАХ}
```

Рамка со штампом генерируется в последнюю очередь – после того, как весь остальной текст на странице скомпонован. Не следует подключать пакеты, которые будут перенастраивать заголовки, например, `titlesec`. Рамки в этом случае нарисованы не будут.

6.5 Оформление иллюстраций

На все рисунки должны быть ссылки в основном тексте записки, причём они должны предшествовать самим рисункам. Название рисунка выравнивается по центру под рисунком. Подписи к рисункам должны быть сделаны шрифтом, размер которого составляет не менее 12 пунктов. Собственные рисунки отмечаются добавлением словосочетания «авторский материал», заимствованные рисунки подписываются со ссылкой на источник литературы в квадратных скобках. Слово «рисунок» перед самой ссылкой сокращать не нужно.

Примеры предложений со ссылкой на рисунок

1. На рисунке ?? изображён заяц Бо, который программирует генератор идеального изображения.
2. Используемый алгоритм (рисунок 2.3) генерации идеального изображения реализован с помощью нейронных сетей глубокого обучения.
3. Представленный на рисунке 3.1 алгоритм распознавания лексем используется во многих интерпретаторах.

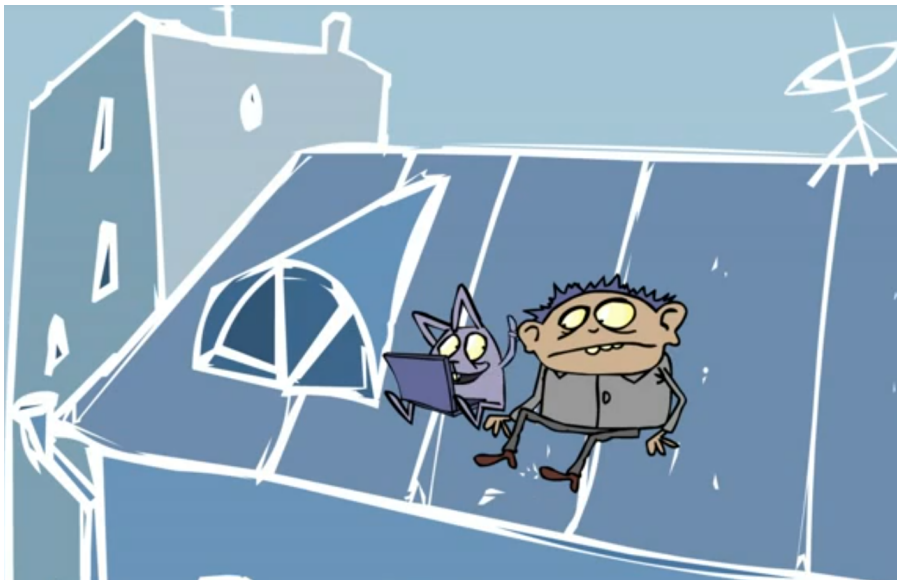


Рисунок 6.2 – Заяц программирует на крыше [1]

Номер рисунка начинается с номера главы, т.е. в первой главе рисунки нумеруются как «Рисунок 1.1; Рисунок 1.2;...», во второй – «Рисунок 2.1; Рисунок 2.2;...» и т.д. Если рисунок занимает более половины страницы, то его необходимо перенести в приложение. Рисунки в приложениях нумеруются прописными буквами русского алфавита, как написано ниже.

Примеры подписей для рисунков в приложениях

- Рисунок А.1 – <Название первого рисунка в Приложении А>. Авторский материал
Рисунок А.2 – <Название второго рисунка в Приложении А>. Авторский материал
Рисунок Б.1 – <Название первого рисунка в Приложении Б>. Авторский материал

Требования к рисункам:

1. Растровые рисунки должны иметь разрешение не менее 300 dpi. Толщина самой тонкой линии после печати – не менее 0.2 мм (как в тетради в клетку).
2. Надписи на рисунках должны быть выполнены на русском языке. Английские названия идентификаторов должны дублироваться в скобках на русском языке. На графиках нужно подписать оси (название и единицы измерения

параметров), изобразить числовую сетку. Надписи на рисунках должны быть хорошо различимы после печати.

3. Основные линии должны быть нарисованы чёрным цветом. Разрешается дополнительно использовать другие цвета, хорошо различимые после чёрно-белой печати. Цветные рисунки должны быть распечатаны цветными красками, если это важно в контексте изложения.

4. Цвет фона на схемах, диаграммах, графиках – белый.

5. Снимки экранов с интегрированных средств разработки не допускаются.

6.6 Оформление таблиц

Название таблицы размещается непосредственно над таблицей, выравнивается по левому краю и имеет вид: «Таблица 1.1 – Название таблицы». Если таблица помещается на листе, то разрывать её не следует. Если таблица не помещается на листе, то её нужно разделить на части. Продолжение таблицы при этом переносится на следующую страницу, в правом верхнем углу которой необходимо написать «Продолжение таблицы 1.1». Строку с заголовком при этом нужно повторить. Ссылки на таблицы в основном тексте оформляются так же, как на рисунки. Колонки таблицы должны быть растянуты на всю рабочую ширину `\textwidth`. Рекомендуется создавать таблицы с использованием пакета `longtable` или аналогичного, который поддерживает многостраничные таблицы.

Пример текста со ссылкой на таблицу

В таблице ?? собраны данные о первых 40 химических веществах из таблицы Менделеева.

Таблица 6.1 – Физические свойства элементов таблицы Менделеева

Элемент	Символ	Название на латыни	Атомный номер	Атомная масса (г/моль)	Плотность при нормальных условиях (г/см ³)
Водород	H	Hydrogenium	1	1,008	0,08988
Гелий	He	Helium	2	4,002602	0,1786
Литий	Li	Lithium	3	6,94	0,534
Бериллий	Be	Beryllium	4	9,0122	1,848

Элемент	Сим- вол	Название на латыни	Атом- ный но- мер	Атомная масса (г/моль)	Плотность при нор- мальных условиях (г/см ³)
Бор	B	Borum	5	10,81	2,34
Углерод	C	Carboneum	6	12,011	2,267
Азот	N	Nitrogenium	7	14,007	0,00125
Кислород	O	Oxygenium	8	15,999	0,001429
Фтор	F	Fluorum	9	18,998403163	0,001696
Неон	Ne	Neon	10	20,1797	0,0008999
Натрий	Na	Natrium	11	22,98976928	0,968
Магний	Mg	Magnesium	12	24,305	1,738
Алюминий	Al	Aluminium	13	26,9815385	2,70
Кремний	Si	Silicium	14	28,0855	2,329
Фосфор	P	Phosphorus	15	30,973761998	1,82
Сера	S	Sulfur	16	32,06	2,067
Хлор	Cl	Chlorum	17	35,45	0,003214
Аргон	Ar	Argon	18	39,948	0,0017837
Калий	K	Kalium	19	39,0983	0,856
Кальций	Ca	Calcium	20	40,078	1,54
Скандий	Sc	Scandium	21	44,955908	2,985
Титан	Ti	Titanium	22	47,867	4,506
Ванадий	V	Vanadium	23	50,9415	6,11
Хром	Cr	Chromium	24	51,9961	7,19
Марганец	Mn	Manganum	25	54,938044	7,43
Железо	Fe	Ferrum	26	55,845	7,874
Кобальт	Co	Cobaltum	27	58,933194	8,90
Никель	Ni	Niccolum	28	58,6934	8,912
Медь	Cu	Cuprum	29	63,546	8,96
Цинк	Zn	Zincum	30	65,38	7,14
Галлий	Ga	Gallium	31	69,723	5,91
Германий	Ge	Germanium	32	72,63	5,323
Мышьяк	As	Arsenicum	33	74,921595	5,72
Селен	Se	Selenium	34	78,971	4,79

Элемент	Сим- вол	Название на латыни	Атом- ный но- мер	Атомная масса (г/моль)	Плотность при нор- мальных условиях (г/см ³)
Бром	Br	Bromum	35	79,904	3,12
Криптон	Kr	Krypton	36	83,798	0,003733
Рубидий	Rb	Rubidium	37	85,4678	1,532
Стронций	Sr	Strontium	38	87,62	2,64
Иттрий	Y	Yttrium	39	88,90584	4,47
Цирконий	Zr	Zirconium	40	91,224	6,52

6.7 Оформление формул

Номер формулы пишется в круглых скобках и выравнивается по правому краю, если на формулу есть ссылка в тексте. Если на формулу ссылки нет, то она не нумеруется. У формул, как и у рисунков, номер начинается с номера главы. Далее после точки следует номер формулы непосредственно внутри главы. Таким образом, в первой главе формулы имеют номера 1.1, 1.2, 1.3 и т.д., во второй – 2.1, 2.2, 2.3 и т.д.

Пример оформления формулы

Формула Планка для спектральной плотности излучения абсолютно чёрного тела имеет вид:

$$I(\nu, T) = \frac{2h\nu^3}{c^2} \cdot \frac{1}{e^{\frac{h\nu}{kT}} - 1}, \quad (6.1)$$

где

- $I(\nu, T)$ – спектральная плотность излучения (Вт/(м²·Гц),
- h – постоянная Планка, равная приблизительно $6,626 \times 10^{-34}$ Дж·с,
- ν – частота излучения (Гц),
- c – скорость света в вакууме (около 3×10^8 м/с),
- k – постоянная Больцмана, равная приблизительно $1,381 \times 10^{-23}$ Дж/К,
- T – абсолютная температура тела (К).

Формула (??) описывает распределение энергии излучения абсолютно чёрного тела в зависимости от частоты ν и температуры T .

Любая формула является частью предложения, поэтому нужно правильно расставлять знаки препинания перед ней и после неё. Если несколько формул следуют друг за другом, то они тоже должны разделяться знаками препинания. В написанном примере формула заканчивается запятой, потому что далее предложение продолжается словом «где». Значит, перед словом «где» абзацный отступ не нужен. Все переменные, которые есть в формуле, должны быть раскрыты в тексте, т.е. после прочтения должно быть понятно, что каждая переменная в формуле означает. Формулы нельзя разрывать на несколько строк где попало. Формулы переносятся на следующую строку только знаками «=», «+», «-» с повторением этого знака в новой строке. Внутри предложений математические переменные тоже должны быть набраны как формулы, т.е. с использованием математического режима $\$ \dots \$$.

Образцы правильно оформленных формул рекомендуется смотреть в книгах по математике или научных журналах. К примеру, очень хорошо оформляются формулы в журнале «Вычислительные методы и программирование», все выпуски которого доступны на сайте

<https://num-meth.ru/index.php/journal/issue/archive>

6.8 Оформление списков

Списки (перечни или перечисления) бывают двух видов – маркированные и нумерованные. В маркированном списке каждый пункт маркируется символом дефиса «-» или закрашенным кругом «●». В нумерованном списке пункты нумеруются строчными буквами русского алфавита или арабскими цифрами. После буквы записывается круглая скобка.

В списках пункты разделяются запятыми, если сами пункты короткие; точками с запятой, если пункты содержат запятые; точками, если пункты содержат целые предложения. Список должен заканчиваться точкой.

Внутри пакета `bstunote` окружения `itemize` (маркированные списки) и `enumerate` (нумерованные списки) настроены на соответствие ГОСТ Р 2.105–2019 [?, раздел 6.7].

Пример маркированного списка

Виды компьютерной графики:

- растровая,
- векторная,
- фрактальная.

Пример нумерованного списка

Любое число, хранимое в оперативной памяти может быть

- а) целым различной разрядности:
 - 1) 8-разрядное (char),
 - 2) 16-разрядное (short),
 - 3) 32-разрядное (int),
 - 4) 64-разрядное (long long);
- б) вещественным различной точности:
 - 1) одинарная точность (float),
 - 2) двойная точность (double);
- в) двоично-десятичным в одном из двух форматов:
 - 1) упакованное,
 - 2) неупакованное.

Правила оформления списков очень хорошо описаны в статье Лотош Л.С. «[Правила оформления перечней](#)» [?].

6.9 Оформление листингов

В основные главы записки можно вставлять небольшие фрагменты программ до 20 строк. Эти фрагменты называются листингами. Если листинг занимает больше 20 строк, то нужно перенести его в приложение и сделать в тексте ссылку на него. Все листинги должны быть подписаны. Название листинга записывается перед листингом и выравнивается по левому краю. Ссылка на листинг должна предшествовать самому листингу. Нельзя вставлять текст программы как картинку. В листингах строки программы должны быть пронумерованы, пустые строки не допускаются. Комментарии должны быть написаны с прописной буквы без грамматических ошибок на русском языке. Обязательные параметры листингов:

- тип шрифта – моноширинный,
- размер шрифта – не более 10 пт.,
- отступ после открытия операторной скобки (по англ. indentation) – два пробела,
- фон – светлый.

Остальные параметры (подсветка, семейство шрифта, цвета и др.) выбираются на усмотрение автора. Для оформления листингов рекомендуется использовать стандартный пакет listings или более современный – minted.

Примеры подписей для листингов

Листинг 1.1 – <Заголовок первого листинга в первой главе>

Листинг 1.2 – <Заголовок второго листинга в первой главе>

Листинг 2.1 – <Заголовок первого листинга во второй главе>

Листинг A.1 – <Заголовок первого листинга в приложении A>

Листинг A.2 – <Заголовок второго листинга в приложении A>

Пример ссылки на листинг

В листинге ?? представлена функция обратной перестановки битов, широко используемая в алгоритмах быстрого преобразования Фурье.

Листинг 6.1 – Реализация обратной перестановки битов на языке Python

```
1 # Обратная перестановка битов
2 def inverse(a, m):
3     r = 0
4     while m:
5         r = 2 * r + a % 2
6         a //= 2
7         m //= 2
8     return r
```

Если в ходе изложения требуется объяснить небольшой фрагмент программы, который невозможно оформить в полноценный листинг, то рекомендуется использовать окружение `\begin{verbatim}... \end{verbatim}`.

Пример использования окружения `verbatim`

Добавим во внутренний цикл функции растеризации вычисление барицентрических координат h_0, h_1, h_2 для текущего пикселя (x, y) :

```
float S = (y1 - y2) * (x0 - x2) + (x2 - x1) * (y0 - y2); // Площадь
float h0 = ((y1 - y2) * (x - x2) + (x2 - x1) * (y - y2)) / S;
float h1 = ((y2 - y0) * (x - x2) + (x0 - x2) * (y - y2)) / S;
float h2 = ((y0 - y1) * (x - x1) + (x1 - x0) * (y - y1)) / S;
```

Тогда координата z точки (текущего пикселя) на треугольнике вычисляется в результате интерполяции координат z трёх вершин треугольника:

```
float Z = h0 * z0 + h1 * z1 + h2 * z2; // Глубина пикселя
```

Названия переменных, методов, классов, полей таблиц БД и прочих идентификаторов, полученных на этапе проектирования собственного ПО должны быть набраны моноширинным шрифтом. Внутри основного текста не нужно записывать круглые скобки после названий методов.

Пример использования моноширинного шрифта внутри текста

Метод `OnScoreMouseMove` вызывается при перемещении мыши в пределах основной рамки `score` и принимает координаты курсора в качестве аргументов `xpos` и `ypos`.

6.10 Оформление списка литературы

Список литературы оформляется в соответствии с ГОСТ Р 7.0.100–2018 [?] и ГОСТ Р 7.0.5–2008 [?]. На все источники в списке литературы должны быть ссылки в тексте. Ссылки на литературные источники должны нумероваться в порядке их появления в тексте, начиная с единицы. Примеры ссылок на источники:

«...в статье [2] описан метод решения...»,

«В работах [3,4] опубликованы результаты...».

Список литературы можно формлять с использованием стандартного окружения `thebibliography`, но лучше использовать более современный пакет `biblatex`. Если литературный источник доступен в интернете, то ссылка на него в самом списке литературы должна быть кликабельной, чтобы легко можно было перейти по ней и открыть.

Ниже представлены примеры оформления литературных источников.

Книга (один автор)

Юров, В.И. *Assembler : учебник для вузов / В. И. Юров.* – 2-е изд. – СПб. : Питер, 2006. – 636 с. : ил.

Книга (два автора)

Шикин, А. В. *Компьютерная графика. Полигональные модели / А. В. Шикин, А. В. Боресков.* – М. : ДИАЛОГ-МИФИ, 2001. – 464 с.

Книга (три автора)

Амосов, А. А. *Вычислительные методы для инженеров : учеб. пособие для студентов вузов, обучающихся по техн. направлениям и специальностям / А. А. Амосов, Ю. А. Дубинский, Н. В. Копченова.* – 2. изд., доп. – Москва : Изд-во МЭИ, 2003. – ISBN 5-7046-0919-8. – EDN QMDWJR.

Статья (один автор)

Брусенцев, А. Г. *Самосопряженность эллиптических дифференциальных операторов в $L_2(G)$ и исправляющие потенциалы / А. Г. Брусенцев // Труды Московского математического общества.* – 2004. – Т. 65. – С. 35-68. – EDN QOLCHI.

Статья (два автора)

Рязанов, Ю. Д. *Построение синтаксических анализаторов на основе син-*

таксических диаграмм с многовходовыми компонентами / Ю. Д. Рязанов, С. В. Назина // Прикладная дискретная математика. – 2022. – № 55. – С. 102-119. – DOI 10.17223/20710410/55/8. – EDN XHAFEV.

Статья (три автора)

Зуев, С. В. Выявление аномалий в потоке с помощью фрактальной размерности графа нейронной сети обработки данных / С. В. Зуев, П. С. Кабелянц, В. М. Поляков // Информационные системы и технологии. – 2021. – № 5(127). – С. 31-38. – EDN PZZKZR.

Статья в интернете (один автор)

Прокопов, Н. А. Моё разочарование в софте / Н. А. Прокопов ; пер. с англ. А. Ализара // Хабр : [электронный ресурс]. – 2018. – 20 сентября. – URL: <https://habr.com/ru/articles/423889/> (дата обращения: 24.12.2025).

6.11 Требования к литературному стилю

Текст ВКР не должен содержать грамматических ошибок (синтаксических, пунктуационных и других). Все предложения должны быть согласованы между собой, быть ясными и понятными. ВКР должна быть написана на русском языке в строгом научном стиле. Разрешается использовать только те слова русского языка, которые являются устоявшимися и определены в словарях русского языка (научно-техническом, толковом).

Нельзя начинать предложение с английского слова, названия идентификатора, математического обозначения или числа.

Перед английскими словами нужно пояснять, что они означают. Например, «метод `beforeEach`», «технология REST API», «API-функция», «HTTP-запрос», «паттерн MVC», «язык Python», «модуль CRM», «переменная `file_name`», «СУБД PostgreSQL».

Пример неграмотно построенного предложения

`addition` находится внутри `TComplex` и написан на C++ с использованием MVC.

Пример грамотно построенного предложения

Метод `addition` находится внутри класса `TComplex` и написан на языке C++ с использованием шаблона MVC.

Слова, которых нет в научно-техническом словаре (а по сути, в русском языке), использовать не разрешается. К примеру, слова «хедер», «футер», «стор», «баг», «продакшн», «бэкап», «батч», «скриншот», «инстанс», «логгер», «краш»,

«эндпоинт», «фронтенд», «дашборд» и т.д. нужно заменять русскоязычными аналогами, используя словарь синонимов. Пояснительная записка к ВКР должна быть оформлена на русском языке в научном стиле, а не звучать как речь настоящего «айтишника», переевшего англицизмов. Если записка содержит англицизмы или сленговые выражения, то она возвращается на полную переработку.

Пример неграмотно построенного предложения

Баги на фронтенде, которые улетят в прод после релиза в сторе, приведут к полному эпик-фейлу, потому что тимлид заапрувил мерж-реквест, скипнув регрессию.

Пример грамотно построенного предложения

Ошибки в клиентской части, которые попадут в рабочую версию после публикации в магазине приложений, приведут к полному провалу, так как руководитель команды утвердил внесение изменений, пропустив этап регрессионного тестирования.

Словари русского языка, в том числе словарь синонимов, доступны на сайте <https://gufo.me>.

Букву «ё» лучше не заменять на «е», иначе теряется смысл некоторых слов. Помним, что от буквы «ё» зависит **передохнем** мы или **передохнём**.

Не следует путать дефис «-» и тире «—» в предложениях.

ЗАКЛЮЧЕНИЕ

СПИСОК ЛИТЕРАТУРЫ

1. *F.R. Stöter, S. Uhlich, A. Liutkus, u Y. Mitsufuji* , «Open-Unmix - A Reference Implementation for Music Source Separation», J. Open Source Softw., т. 4, вып. 41, с. 1667, 2019, doi: 10.21105/joss.01667.
2. *Uhlich S. et al.* Open-Unmix: A Deep Neural Network Reference Implementation for Music Source Separation – GitHub. 2019. – URL: <https://github.com/sigsep/open-unmix-pytorch>
3. **ГОСТ 19.201-78.** Техническое задание. Требования к содержанию и оформлению. – Введ. 1980-01-01. – Москва : Стандартинформ, 2010. – 4 с. – URL: <https://files.stroyinf.ru/Data2/1/4294850/4294850109.pdf>
4. **ГОСТ 19.201-78.** Техническое задание. Требования к содержанию и оформлению. – Введ. 1980-01-01. – Москва : Стандартинформ, 2010. – 4 с. – URL: <https://files.stroyinf.ru/Data2/1/4294850/4294850109.pdf>
5. **ГОСТ 2.105–2019.** Единая система конструкторской документации. Общие требования к текстовым документам. – Введ. 2020-02-01. – М. : Стандартинформ, 2021. – 40 с. – URL: <https://gostexpert.ru/gost/gost-2.105-2019>
6. **ГОСТ Р 2.104-2023.** Единая система конструкторской документации. Основные надписи. – Введ. 2024-03-01. – М. : Российский институт стандартизации, 2024. – 20 с. – URL: <https://files.stroyinf.ru/Data/816/81679.pdf>
7. *Лотош, Л. С.* Правила оформления перечней / Л. С. Лотош // Делопр-изводство и документооборот на предприятии : [электронный ресурс]. – 2007. – № 5. – URL: <https://delo-press.ru/journals/documents/oformlenie-dokumentov/34821-pravila-oformleniya-perechney/> (дата обращения: 24.12.2025).
8. **ГОСТ Р 7.0.100–2018.** Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Введ. 2019-07-01. – М. : Стандартинформ, 2018. – 73 с. – URL: <https://gostexpert.ru/gost/gost-7.0.100-2018>
9. **ГОСТ Р 7.0.5–2008.** Библиографическая ссылка. Общие требования и правила составления. – Введ. 2009-01-01. – Москва : Стандартинформ, 2008. – 23 с. – URL: <https://gostexpert.ru/gost/gost-7.0.5-2008>

ПРИЛОЖЕНИЕ А

Блок-схемы основных алгоритмов быстрого преобразования Фурье

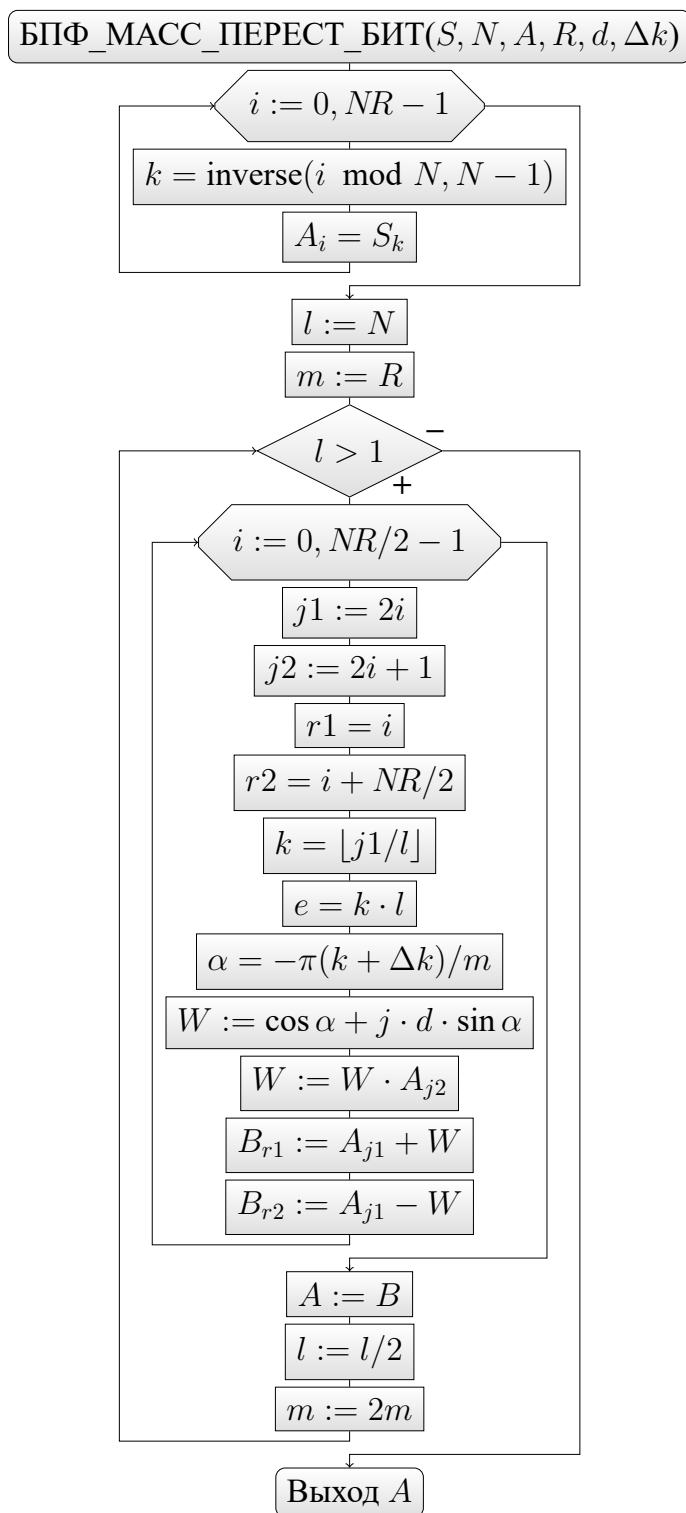


Рисунок А.1 – Алгоритм БПФ с прореживанием по времени, обратной перестановкой битов и дополнительным массивом. Авторский материал

ПРИЛОЖЕНИЕ Б

Листинги разработанных подпрограмм для выполнения быстрого преобразования Фурье

Листинг Б.1 – Реализация алгоритма БПФ с прореживанием по времени, обратной перестановкой битов и дополнительным массивом на языке C++

```
1  void БПФ С ПЕРЕСТ БИТОВ И ВТОРЫМ МАССИВОМ(complex<double>* S, int N, complex<
   ↪ double>* A, int R, int d, double dk=0)
2  {
3      int l, i, j1, j2, r1, r2, m, NR = N*R, k, e;
4      double angle;
5      for (i=0; i < NR; i++)
6      {
7          k = inverse(i % N, N-1);
8          A[i] = S[k];
9      }
10     complex<double>* B = new complex<double>[NR], W;
11     l = N; m = R;
12     while(l > 1)
13     {
14         for (i=0; i < NR/2; i++)
15         {
16             j1 = 2*i; j2 = 2*i + 1;
17             r1 = i;   r2 = i + NR/2;
18             k = j1 / l;
19             e = k * l;
20             angle = -PI * (k + dk) / m;
21             W = polar<double>(1, d * angle);
22             W = W * A[j2];
23             B[r1] = A[j1] + W;
24             B[r2] = A[j1] - W;
25         }
26         memcpy(A, B, sizeof(complex<double>) * NR);
27         l = l >> 1;
28         m = m << 1;
29     }
30     delete []B;
31 }
```

СЛУЖЕБНАЯ ИНФОРМАЦИЯ

Список иллюстраций

**Список картинок нужен только для прохождения нормоконтроля.
В диплом он не вшивается!**

СЛУЖЕБНАЯ ИНФОРМАЦИЯ

Список таблиц

**Список таблиц нужен только для прохождения нормоконтроля.
В диплом он не вшивается!**

СЛУЖЕБНАЯ ИНФОРМАЦИЯ

Листинги

6.1	Реализация обратной перестановки битов на языке Python . . .	39
Б.1	Реализация алгоритма БПФ с прореживанием по времени, обратной перестановкой битов и дополнительным массивом на языке C++	46

**Список листингов нужен только для прохождения нормоконтроля.
В диплом он не вшивается!**